

VersaSlot: Efficient Fine-grained FPGA Sharing with Big.Little Slots and Live Migration in FPGA Cluster

Jianfeng Gu, Hao Wang, Xiaorang Guo, Martin Schulz and Michael Gerndt

Technical University of Munich



Leibniz-Rechenzentrum
der Bayerischen Akademie der Wissenschaften

SPONSORED BY



Outline

- Background
- Motivation
- System Design
- Evaluation
- Conclusion



Background & Motivation



SPONSORED BY



Background - FPGA Cloud

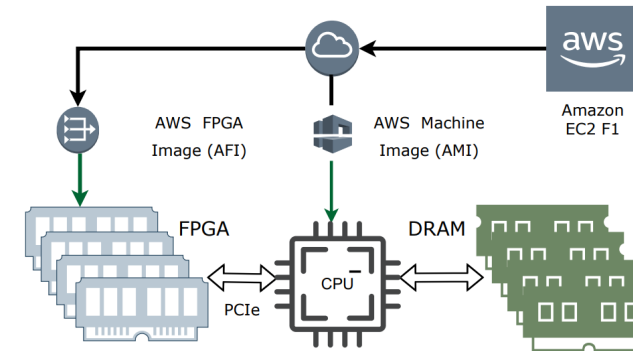
• FPGA as a Service in the Cloud

Advantages

- Efficient application acceleration with higher Flexibility;
- Lower power consumption;
- Customized and on-demand applications with Quality of Service;

Popularity

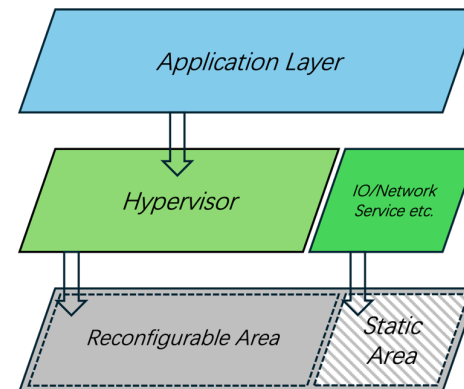
- Microsoft Catapult, Amazon EC2 F1, Alibaba FPGA Cloud;
- OSDI20 Coyote [2], ASPLOS22 XPU-Shim [3], ISCA23 Nimblock [4], and etc.



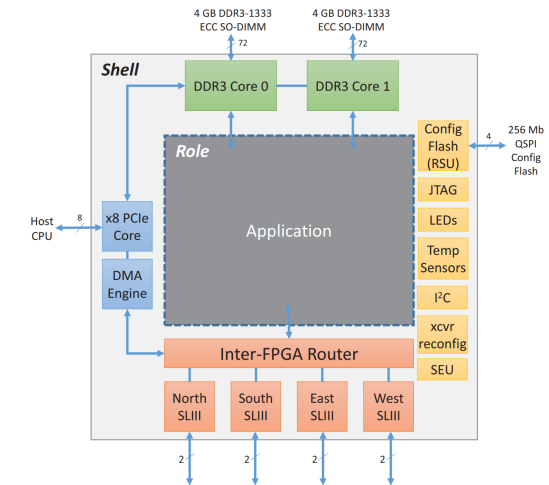
Amazon EC2 F1 FPGA Cluster

• Time-multiplexing FPGA Sharing

- FPGA Virtualization: 3 tiers;
- Allocate the entire FPGA to a single application;
- Full fabric reconfiguration;
- Large context switch overhead and resource under-utilization;



FPGA Virtualization Mode



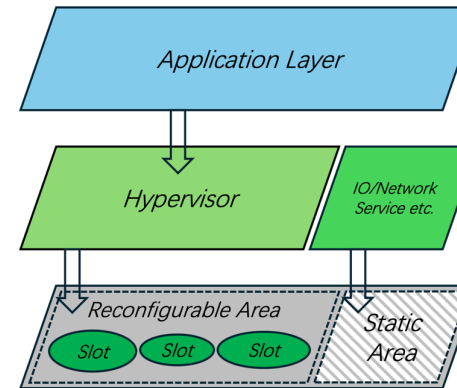
Intel Catapult [1]



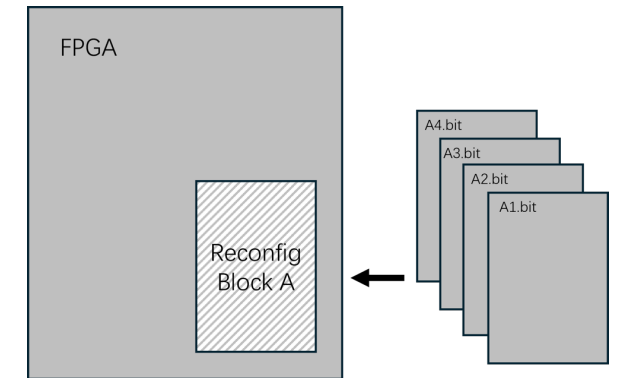
Background - FPGA Sharing

- **Spatial FPGA Sharing**

- Split the **Reconfigurable Area** into multiple **slots**;
- **Slot**: A Partial reconfigurable region;
- **Task**: A portion of an application, the basic execution unit for a slot.
- Based on Dynamic Partial Reconfiguration;



Slot: Partial Reconfiguration Region



Dynamic Partial Reconfiguration to load bitstream

- **Dynamic Partial Reconfiguration (DPR)**

- *Dynamic Function eXchange (DFX)*;
- Perform *Partial Reconfiguration (PR)* on a slot while other slots continue running;
- **PCAP (Processor Configuration Access Port)**: load applications' partial bitstreams;
- **Limitation:**

PCAP can only perform **serial bitstream loading** and **block the corresponding CPU** to confirm a successful loading.

Motivation

- **Problem 1: Mutual PR contention**

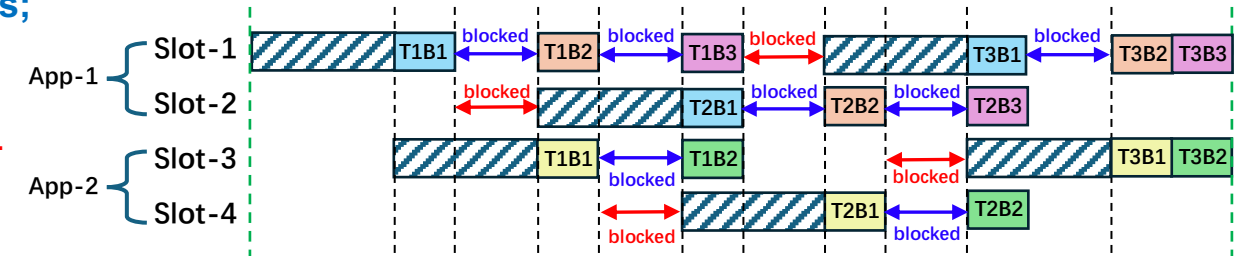
- The Partial Reconfiguration (PR) Module is limited to **serial bitstream loading**;
- **Frequent PR contention among applications**;
- **More applications, more severe PR contention**;

- **Problem 2: Mutual task execution blocking**

- The Partial Reconfiguration (PR) process **blocks the corresponding CPU execution**;
- Previous works relied on a single CPU core for scheduling;
- **PR operations block the launching of new application tasks**;

Causing:

- **Highly prolonged and unpredictable application response times.**
- **Disrupts applications' original task pipelines;**



- **Problem 3: Strict constraints on task partitioning**

- Easily cause resource **over-subscription** and **underutilization** within slots when partitioning the applications to tasks;

System Design



SPONSORED BY



System Design - VersaSlot

- **Hypervisor in the PS**

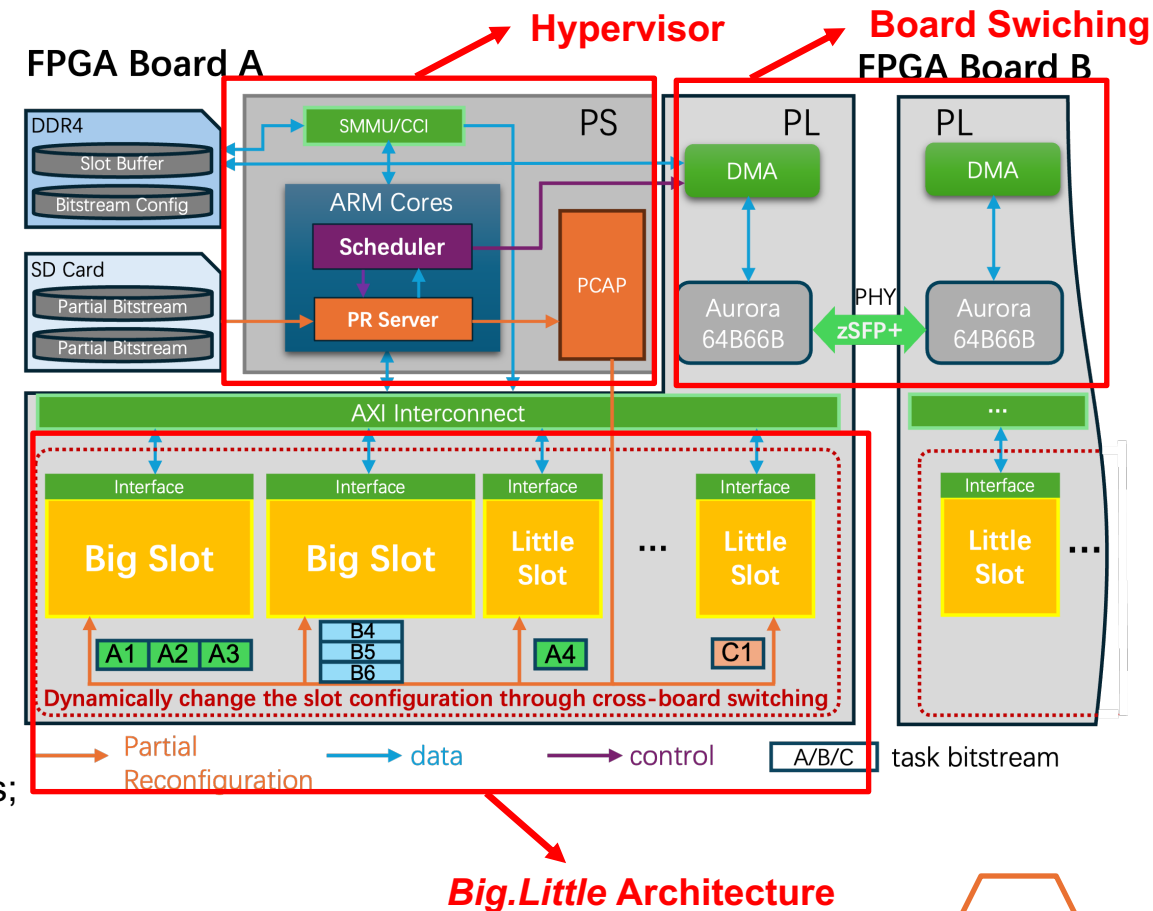
- **Decouple** the PR process and Scheduling with dual cores;
- **Scheduler**;
- **PR Server**;
- PCAP: load partial bitstream;
- **Solve**: Mutual task execution blocking problem;

- **Big.Little Slots in the PL**

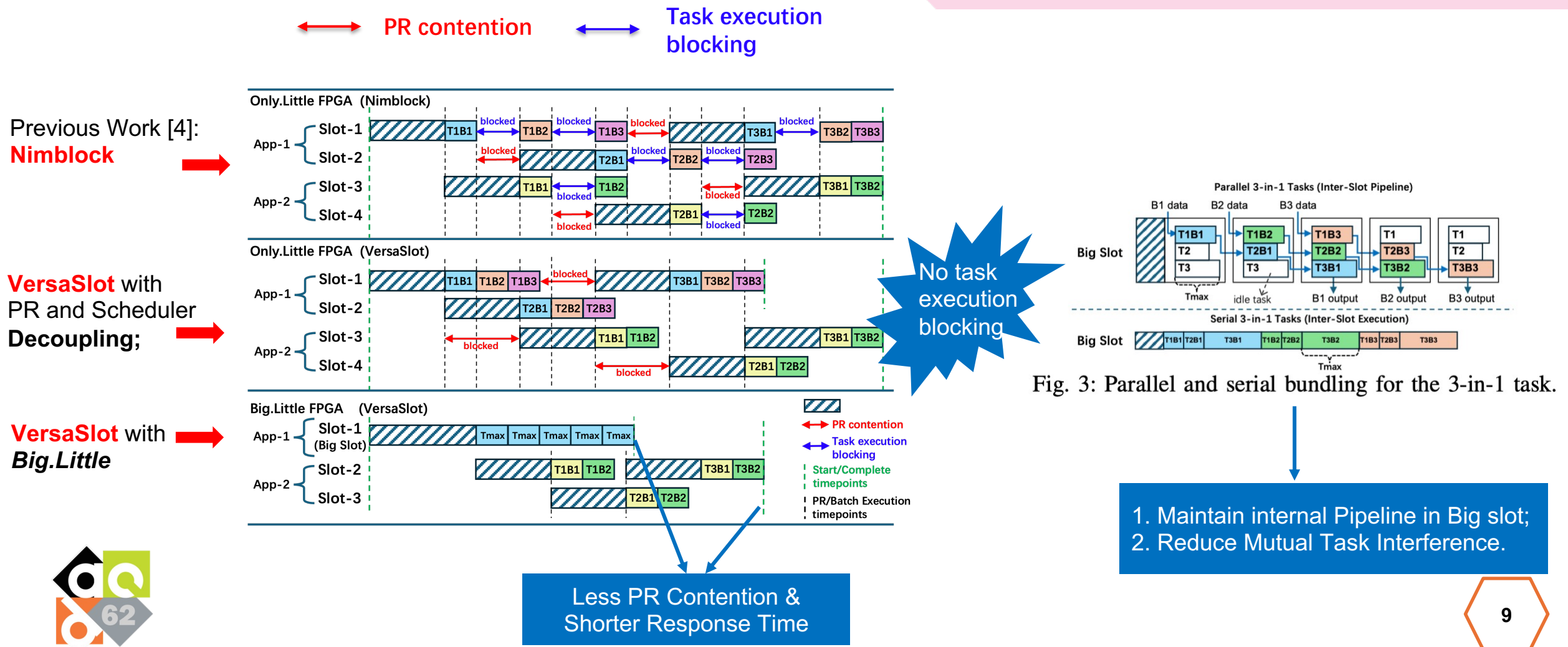
- **Big.Little Architecture**;
- **Idea**: Bundle multiple tasks into Big slots to reduce PR frequency;
- Dynamic batching processing;
- **Mitigate**: Mutual PR contention;

- **Cross-board Switching Module**

- Switching between boards with **Big.Little** slots and **Only.Little** slots;
- Aurora IP core to transfer tasks;



System Design – *Big.Little* Architecture



System Design – Allocation & Scheduling

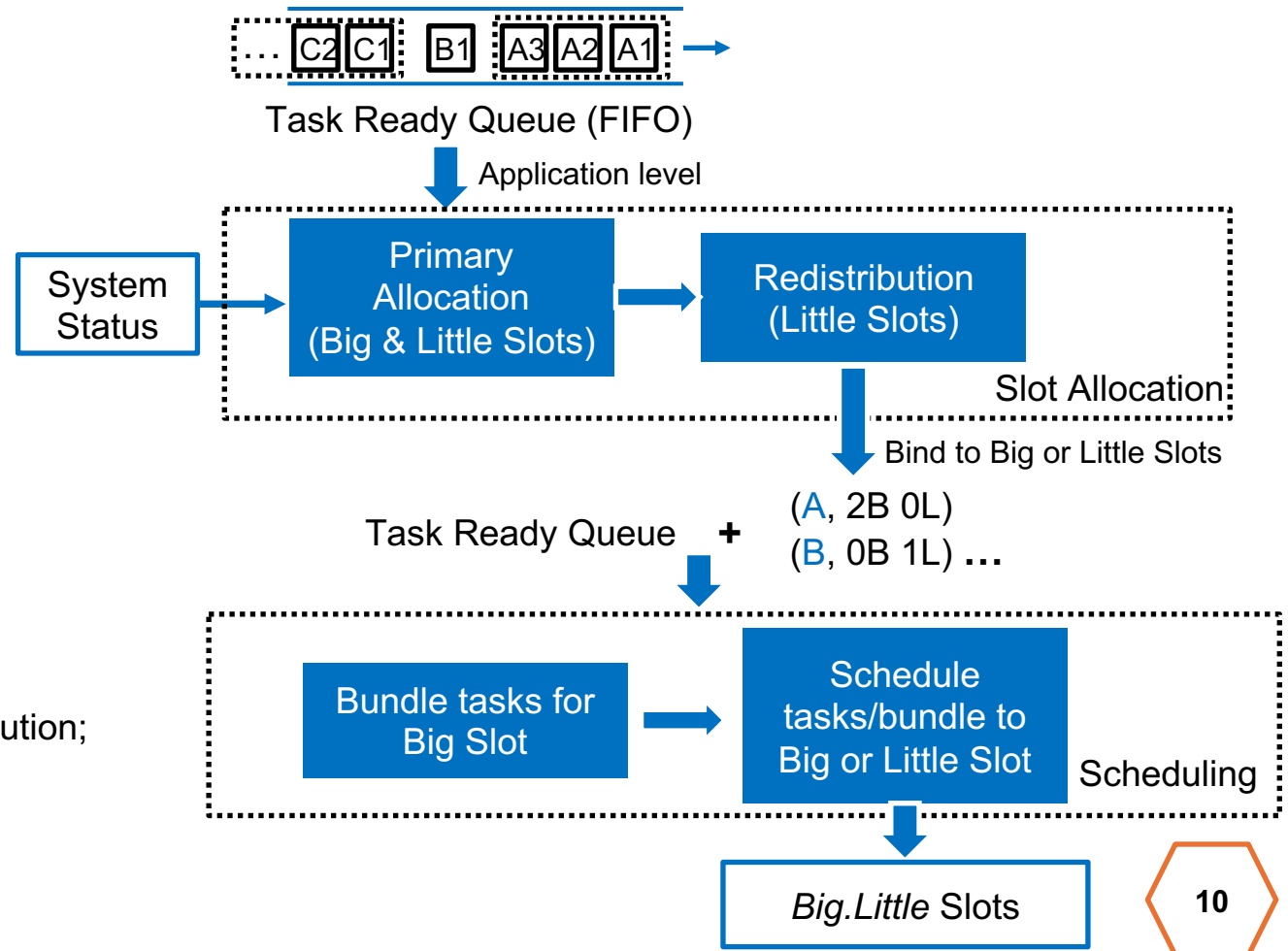
Heterogeneous *Big.Little* slot architecture introduces greater complexity in resource allocation and scheduling.

• Slot Allocation Algorithm

- **Primary Allocation:**
 - Big Slots First (to reduce PR contention);
 - Guarantee Fairness;
- **Redistribution:**
 - Based on available Little slots;
 - Maximize slot usage (FCFS);
- **Binding and Rebinding:**
 - Unscheduled applications back to Ready Queue;
 - To avoid Busy Big Slots but Little Slot Starvation;

• On-board Scheduling Algorithm

- Online bundling tasks of an application for Big Slot execution;
- Schedule tasks to the corresponding Big or Little Slots.



System Design - Cross-board Switching

- **Cross-boarding Switching**

- Switching among *Big.Little* FPGA and *Only.Little* FPGA;
- ***Big.Little*** -> Mitigate PR contention;
- ***Only.Little*** -> Share an FPGA with more applications.

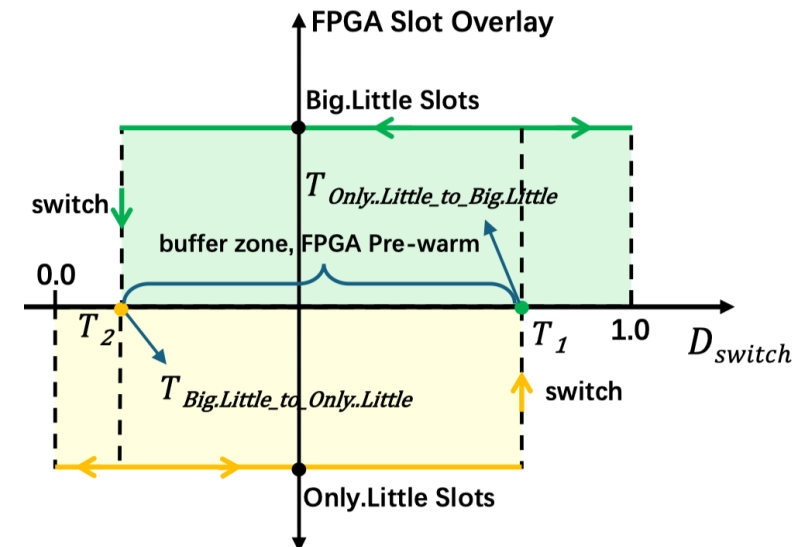
$$D_{\text{switch}} = \frac{N_{\text{blocked_tasks}}}{N_{\text{PR}}} \times \frac{N_{\text{apps}}}{N_{\text{batch}}}, \quad (0 < D_{\text{switch}} < 1)$$

- **Performance Degradation Metric – D_{switch}**

- Evaluate the degree of system PR contention;

- **Live Migration Loop**

- **Two User-configurable thresholds:**
High $D_{\text{switch}} \rightarrow \text{Big.Little}$ FPGA; Low $D_{\text{switch}} \rightarrow \text{Only.Little}$ FPGA;
- **Seamless migration with low overhead;**
- **Buffer zone for the new FPGA Pre-warming to avoid cold-start;**



Experiment & Evaluation



SPONSORED BY



Experiment

- **Experiment Setup**

- Two Xilinx UltraScale+ FPGAs (ZCU216) FPGAs to build a cluster;
- GT transceivers (zSFP+) to connect FPGA boards;

- **Benchmark**

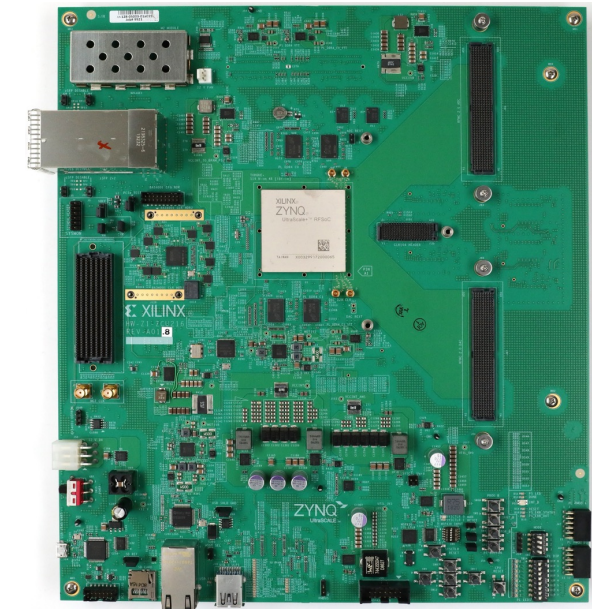
- Same as previous work *ISCA23 Nimblock* [4];

Applications	3D Rendering	LeNet	Image Compression	Optical Flow	AlexNet
Number of tasks	3	6	6	9	6

- **Workload Setting**

- Randomly generated application workloads (10 sequences, 20 apps/sequence)
- Random batch size (5-30) and **arrival intervals** as below:

Real Time	Stress	Standard	Loose
50 ms	150 – 200 ms	1500 – 2000 ms	5000 ms



2 x Xilinx Zynq™ UltraScale+™
RFSoc ZCU216 FPGAs

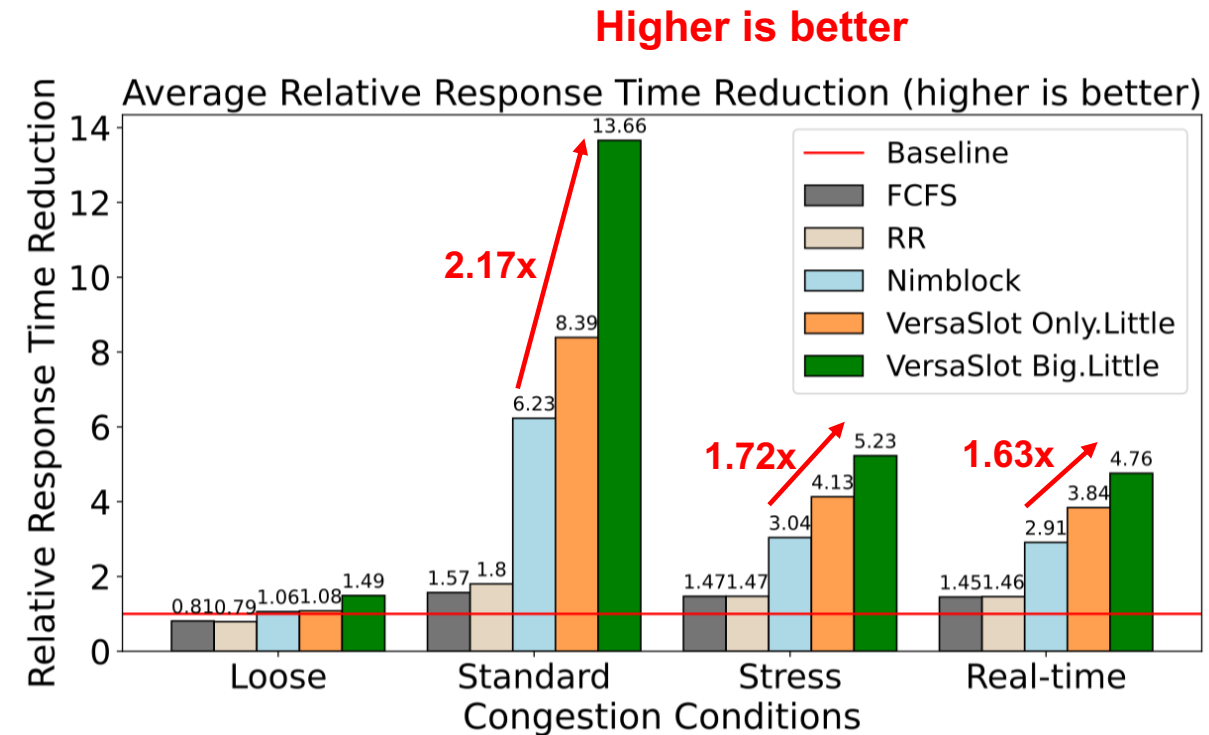
Evaluation

- Compared with the four existing scheduling algorithms/systems

- **Baseline**: traditional exclusive temporal multiplexing [5];
- **FCFS** : First-come-first-served spatio-temporal sharing;
- **RR**: Round-robin [6];
- **Nimblock**: The **state-of-the-art** system [4];

- Relative Response Time Reduction

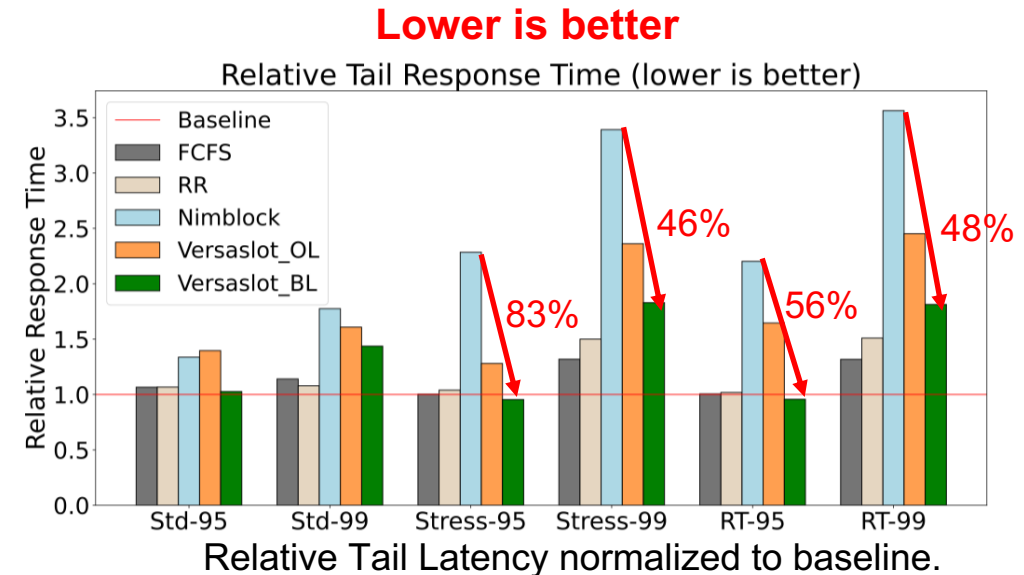
- **VersaSlot Big.Little** outperforms all other methods across all congestion conditions;
 - **Maximum 13.66x reduction** compared to the baseline;
- **Big.Little vs. Nimblock**:
 - **Reduce 2.17x (standard)**, 1.72x (stress), 1.63x (real-time);
 - **Efficiency of Big.Little architecture and dual-core scheduling**;
- **Big.Little vs. Only.Little**:
 - Reduce response time **by 63% (standard)**, 27% (stress), and 24% (real-time);
 - **Verify the effectiveness of Big.Little architecture in alleviating PR contention.**



Evaluation

• Relative Tail Response Time

- **VersaSlot *Big.Little*** outperforms all other methods across all congestion conditions in **P95 tail latency**;
- ***Big.Little* vs. *Nimblock*:**
 - Stress workload: P95 and P99 improve by **83% and 46%**;
 - Real-time workload: P95 and P99 improve by **56% and 48%**;
- ***Big.Little* vs. *Baseline*, *FCFS*, and *RR*:**
 - *Big.Little* shows a very slight increase in P99 tail latency, but negligible;

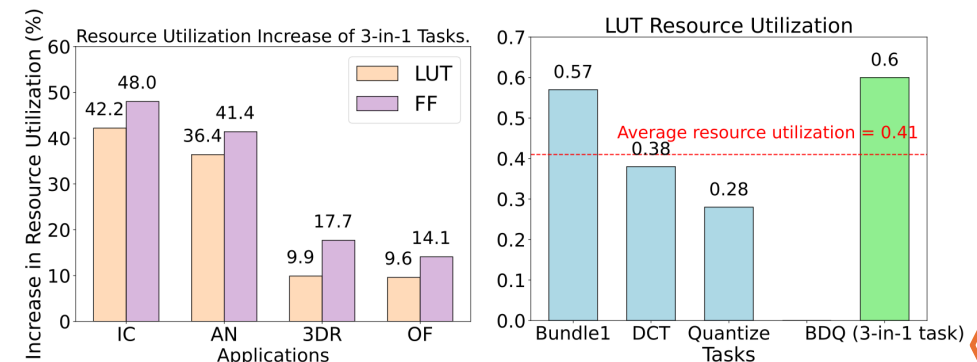


• Resource Utilization

- ***Big.Little* vs. *Only.Little* and *Nimblock*:**
 - LUT and FF utilization increase by **35% and 29%** on average;
 - Task partitioning is normally based on synthesis resource usage; eg. Bundle1 LUT: 0.98 (synthesis) -> 0.57 (implementation);



Task bundling can reduce resource waste.



Evaluation

- **Cross-board Switching**

- **Only.Little to Big.Little switching:**

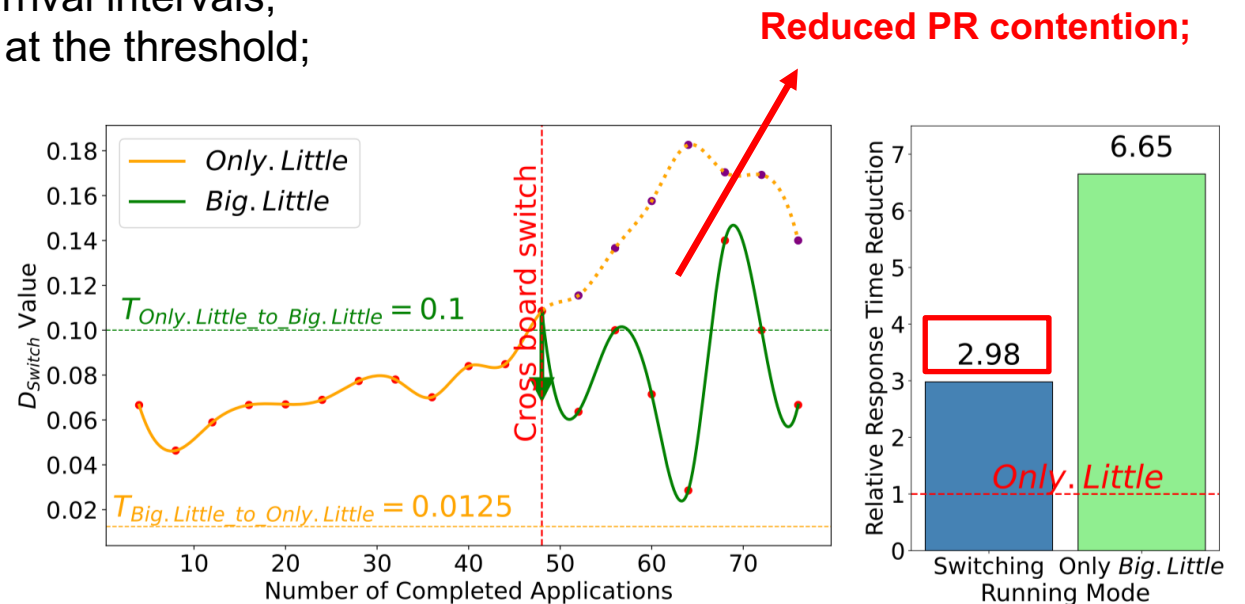
- Three long workloads, 80 applications, standard arrival intervals;
 - D_{switch} triggers switch from *Only.Little* to *Big.Little* at the threshold;
 - Reduced PR contention;

- **Cross-board Switching vs. Only.Little FPGAs**

- Reduce the average response time up to nearly **3x**;

- **Switching overhead:**

- Average switching latency: **1.13ms**.



Relative response time with switching

Conclusion

- **VersaSlot with *Big.Little* Slots**

- An efficient spatio-temporal FPGA sharing system with novel ***Big.Little Slot*** architecture to resolve partial reconfiguration contention and task blocking problems;

- **Slot Allocation and Scheduling Algorithms**

- Efficient slot allocation and scheduling algorithms for the heterogeneous ***Big.Little*** architecture;

- **Cross-board Switching and Live Migration**

- Seamless Cross-board Switching and Live Migration mechanism to maximize FPGA multiplexing across the cluster;

- **Performance**

- Achieve up to **13.66x** lower average response time than the baseline, and up to **2.19x** average response time improvement over the state-of-the-art system;
- Improve the LUT and FF resource utilization by **35% and 29%** on average





Jianfeng Gu

Chair of Computer Architecture and Parallel System
Technical University of Munich



Leibniz-Rechenzentrum
der Bayerischen Akademie der Wissenschaften



AI



Security



Systems



EDA



Design



THE CHIPS
TO SYSTEMS
CONFERENCE

SPONSORED BY

